

CHAPTER

11

Information Retrieval and Retrieval-Augmented Generation

On two occasions I have been asked,—“Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?” ... I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question. Babbage (1864)

People need to know things. So pretty much as soon as there were computers we were asking them questions. By 1961 there was a system to answer questions about American baseball statistics like “How many games did the Yankees play in July?” (Green et al., 1961). Even fictional computers in the 1970s like Deep Thought, invented by Douglas Adams in *The Hitchhiker’s Guide to the Galaxy*, answered “the Ultimate Question Of Life, The Universe, and Everything”.¹ And because so much knowledge is encoded in text, systems were answering questions at human-level performance even before LLMs: IBM’s Watson system won the TV game-show *Jeopardy!* in 2011, surpassing humans at answering questions like:

WILLIAM WILKINSON’S “AN ACCOUNT OF THE PRINCIPALITIES OF WALLACHIA AND MOLDOVIA” INSPIRED THIS AUTHOR’S MOST FAMOUS NOVEL.²

It follows naturally, then, that an important function of large language models is to fill **human information needs**. And since a lot of information is online, finding the information that fills our needs is closely related to web information retrieval, the task performed by search engines. Indeed, the distinction is becoming ever more fuzzy, as modern search engines are integrated with large language models.

factoid questions

Consider some simple information needs, for example **factoid questions** that can be met with facts expressed in short texts like the following:

- (11.1) Where is the Louvre Museum located?
- (11.2) Where does the energy in a nuclear explosion come from?
- (11.3) How to get a script l in latex?

To get an LLM to answer these questions, we can just prompt it! For example a pretrained LLM that has been instruction-tuned on answering questions (instruction-tuning is in Chapter 10) could directly answer the following question

Where is the Louvre Museum located?

by performing conditional generation given this prefix, and take the response as the answer. This works because large language models have processed a lot of facts in their pretraining data, including the location of the Louvre, and have encoded this information in their parameters. Factual knowledge of this type seems to be stored in the connections in the very large feedforward layers of transformer models (Geva et al., 2021; Meng et al., 2022).

¹ The answer was 42, but unfortunately the question was never revealed.

² The answer, of course, is ‘Who is Bram Stoker’, and the novel was *Dracula*.

Simply prompting an LLM is useful for many generation tasks, including those involving facts. But the fact that knowledge is stored in the feedforward weights of the LLM leads to a number of problems with prompting as a method for correctly generating factual texts or answers.

hallucinate

The first and main problem is that LLMs are often incorrect when generating answers or other texts about facts! Large language models **hallucinate**. A hallucination is a response that is not faithful to the facts of the world. That is, when asked questions, large language models sometimes make up answers that sound reasonable. For example, [Dahl et al. \(2024\)](#) found that when asked questions about the legal domain (like about particular legal cases), large language models hallucinated from 69% to 88% of the time! LLMs sometimes give incorrect factual responses even when the correct facts are stored in the parameters; this seems to be caused by the feedforward layers failing to recall the knowledge stored in their parameters ([Jiang et al., 2024](#)).

calibrated

And it's not always possible to tell when language models are hallucinating, partly because LLMs aren't well-**calibrated**. In a **calibrated** system, the confidence of a system in the correctness of its answer is highly correlated with the probability of an answer being correct. So if a calibrated system is wrong, at least it might hedge its answer or tell us to go check another source. But since language models are not well-calibrated, they often give a very wrong answer with complete certainty ([Zhou et al., 2024](#)).

A second problem with meeting user information needs with simple prompting methods is that prompting a large language model to answer from its pretrained parameters doesn't allow us to ask questions about proprietary data. We would like to use language models to help with user information needs about proprietary data like personal email. Or for the healthcare application we might want to apply a language model to medical records. Or a company may have internal documents that contain answers for customer service or internal use. Or legal firms need to ask questions about legal discovery from proprietary documents. None of this data (hopefully) was in the large web-based corpora that large language models are pretrained on.

A final issue with using large language models to answer knowledge questions is that they are static; they were pretrained once, at a particular time. This means that LLMs cannot talk about rapidly changing information (like something that happened last week) since they won't have up-to-date information from after their release data.

RAG
information
retrieval

One solution to all these problems with simple prompting for generating factual text is to give a language model external sources of knowledge, for example proprietary texts like medical or legal records, personal emails, or corporate documents, and to use those documents in answering questions. This method is called **retrieval-augmented generation** or **RAG**, and that is the method we will focus on in this chapter. In RAG we use **information retrieval (IR)** techniques to retrieve documents that are likely to have information that might help answer the question. Then we use a large language model to **generate** an answer given these documents.

Basing our answers on retrieved documents can solve some of the problems with using simple prompting to answer questions. First, it helps ensure that the answer is grounded in facts from some curated dataset. And the system can give the user the answer accompanied by the context of the passage or document it came from. This information can help users have confidence in the accuracy of the answer (or help them spot when it is wrong!). And these retrieval techniques can be used on any proprietary data we want, such as legal or medical data for those applications.

We'll begin by introducing information retrieval, the task of choosing the most relevant document from a document set given a user's query expressing their information need. We'll see the classic method based on cosines of sparse tf-idf vectors, modern neural 'dense' retrievers based on instead representing queries and documents neurally with BERT or other language models. We then introduce the retrieval-augmented generation paradigm.

Finally, we'll discuss various datasets with questions and answers that can be used for finetuning LLMs in instruction tuning and for use as benchmarks for evaluation.

11.1 Information Retrieval

information
retrieval
IR

Information retrieval or **IR** is the name of the field encompassing the retrieval of all manner of media based on user information needs. The resulting IR system is often called a **search engine**. Our goal in this section is to give a sufficient overview of IR to see its application to large language models meeting user information needs. Readers with more interest specifically in information retrieval should see the Historical Notes section at the end of the chapter.

ad hoc retrieval

The IR task we consider is called **ad hoc retrieval**, in which a user poses a **query** to a retrieval system, which then returns an ordered set of **documents** from some **collection**. A **document** refers to whatever unit of text the system indexes and retrieves (web pages, scientific papers, news articles, or even shorter passages like paragraphs). A **collection** refers to a set of documents being used to satisfy user requests. A collection can mean the entire web, in which case we are doing **web search**. But a collection can also be a smaller corporate repo, or even a set of documents used by one person. **term** refers to a word in a collection, but it may also include phrases. Finally, a **query** represents a user's information need expressed as a set of terms.

document

collection

web search

term

query

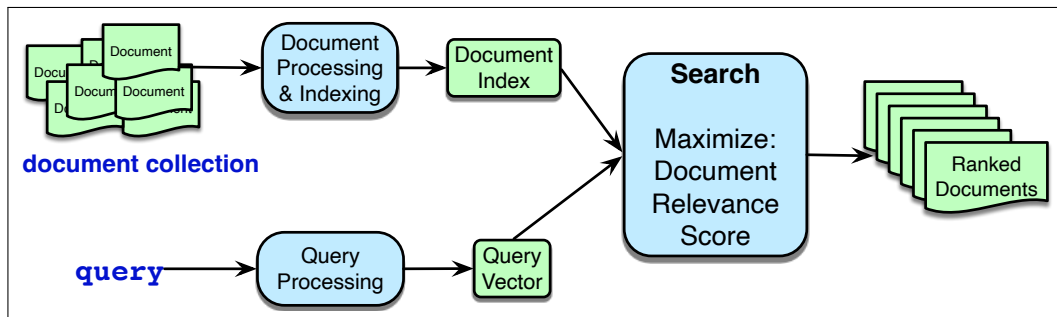


Figure 11.1 The architecture of an ad hoc IR system. Document ranking is based on computing a score for each candidate document given the query, expressing how **relevant** it is likely to be to meet the users information need. There are two classes of IR systems, based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. These two kinds of retrieval differ in the details of the indexing and scoring mechanisms.

The high-level architecture of an ad hoc retrieval engine is shown in Fig. 11.1. This figure abstracts over the two classes of IR systems, which are based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. In sparse retrieval, we represent documents and queries with **count vectors**, weighted by tf-idf or BM25. In dense retrieval, we represent

documents and queries with **embeddings**, computed from language models (either encoder or decoder models). We'll discuss sparse retrieval in the rest of this section, and turn to dense retrieval in Section 11.3.

11.1.1 Representing documents as vectors

vector space
model

In the **vector space model** of information retrieval (Salton, 1971) a document is represented as a vector of counts of the words it contains.

bag of words

We sometimes call this kind of model a **bag-of-words** model. Fig. 11.2 shows the intuition: we are representing a text document as if it were a **bag of words**, that is, an unordered set of words with their position ignored, keeping only their frequency in the document. In the example in the figure, instead of representing the word order in all the phrases like “It manages to be whimsical and romantic”, we simply note that the word *it* occurred 5 times in the entire excerpt, the words *love*, *recommend*, and *movie* once, and so on.

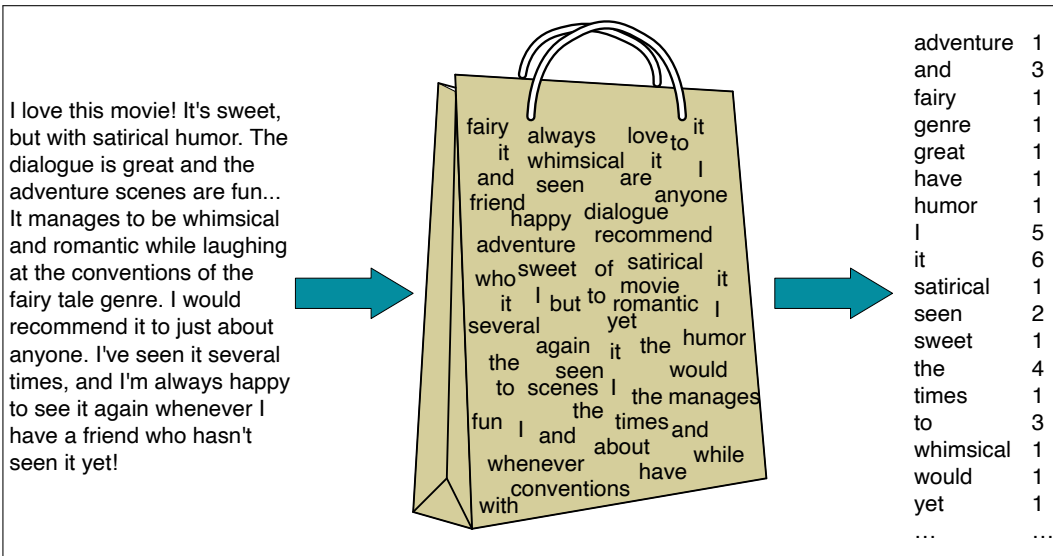


Figure 11.2 Intuition of the classic vector space model applied to a single document. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

We could thus imagine representing the document in Fig. 11.2 as the vector [1 3 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1] (if we limited ourselves to these 18 dimensions and ignored all the other words in English).

term-document
matrix

More generally, we can represent a set of documents as a **term-document matrix** in which each row represents a word in the vocabulary and each column represents a document from some collection of documents. Fig. 11.3 shows a small selection from a term-document matrix showing the occurrence of four words in four plays by Shakespeare. Each cell in this matrix represents the number of times a particular word (defined by the row) occurs in a particular document (defined by the column). Thus *fool* appeared 58 times in *Twelfth Night*.

A document is represented as a count vector, a column in Fig. 11.4. In the example in Fig. 11.4, we've chosen to make the document vectors of dimension 4, just so they fit on the page; in real term-document matrices, the document vectors would have dimensionality $|V|$, the vocabulary size. The first dimension for both these vectors corresponds to the number of times the word *battle* occurs, and we can

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 11.3 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

compare each dimension, noting for example that the vectors for *As You Like It* and *Twelfth Night* have similar values (1 and 0, respectively) for the first dimension.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 11.4 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each document is represented as a column vector of length four.

Since 4-dimensional spaces are hard to visualize, Fig. 11.5 shows a visualization of the four document vectors in two dimensions; we've arbitrarily chosen the dimensions corresponding to the words *battle* and *fool*.

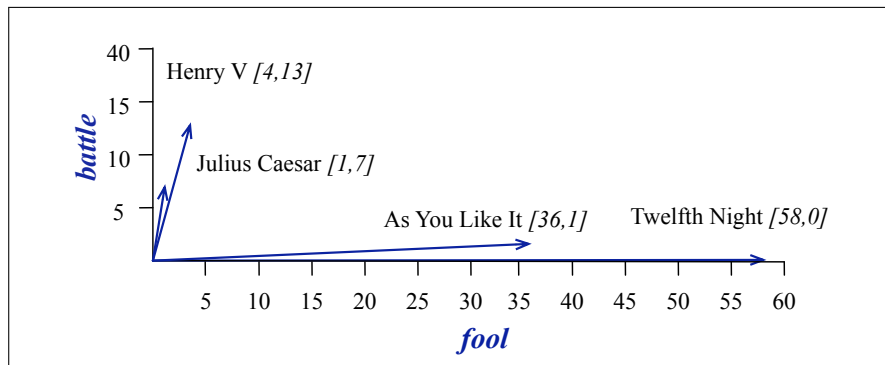


Figure 11.5 A spatial visualization of the document vectors for the four Shakespeare play documents, showing just two of the dimensions, corresponding to the words *battle* and *fool*. The comedies have high values for the *fool* dimension and low values for the *battle* dimension.

Two documents that are similar will tend to have similar words, and if two documents have similar words their column vectors will tend to be similar. The vectors for the comedies *As You Like It* [1,114,36,20] and *Twelfth Night* [0,80,58,15] look a lot more like each other (more fools and wit than battles) than they look like *Julius Caesar* [7,62,1,2] or *Henry V* [13,89,4,3].

11.1.2 Term weighting: tf-idf and BM25

term weight
BM25

In fact, in IR, we don't use raw word counts like [1 114 36 20] for *As You Like It*, or [1 3 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1] for the document in Fig. 11.2. Instead we compute a **term weight** for each document word. Two term weighting schemes are common: **tf-idf** and a variant of tf-idf called **BM25**.

Tf-idf (the '-' here is a hyphen, not a minus sign) is the product of two terms, the term frequency **tf** and the inverse document frequency **idf**.

The **term frequency** term tells us how frequent the word is; words that occur more often in a document are likely to be informative about the document's contents. We usually use the $\log_{10}$ of the word frequency, rather than the raw count. The intuition is that a word appearing 100 times in a document doesn't make that word 100 times more likely to be relevant to the meaning of the document. We also need to do something special with counts of 0, since we can't take the log of 0.³ So if we define $\text{count}(t, d)$ as the raw count of term t in document d , then $\text{tf}_{t,d}$, the tf of term t in document d is

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (11.4)$$

If we use log weighting, terms which occur 0 times in a document would have $\text{tf} = 0$, 1 times in a document $\text{tf} = 1 + \log_{10}(1) = 1 + 0 = 1$, 10 times in a document $\text{tf} = 1 + \log_{10}(10) = 2$, 100 times $\text{tf} = 1 + \log_{10}(100) = 3$, 1000 times $\text{tf} = 4$, and so on.

The **document frequency** df_t of a term t is the number of documents it occurs in. Terms that occur in only a few documents are useful for discriminating those documents from the rest of the collection; terms that occur across the entire collection aren't as helpful. The **inverse document frequency** or **idf** term weight (Sparck Jones, 1972) is defined as:

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t} \quad (11.5)$$

where N is the total number of documents in the collection, and df_t is the number of documents in which term t occurs. The fewer documents in which a term occurs, the higher this weight; the lowest weight of 0 is assigned to terms that occur in every document.

Here are some idf values for some words in the corpus of Shakespeare plays, ranging from extremely informative words that occur in only one play like *Romeo*, to those that occur in a few like *salad* or *Falstaff*, to those that are very common like *fool* or so common as to be completely non-discriminative since they occur in all 37 plays like *good* or *sweet*.⁴

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

The **tf-idf** value for word t in document d is then the product of term frequency $\text{tf}_{t,d}$ and IDF:

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \cdot \text{idf}_t \quad (11.6)$$

³ We can also use this alternative formulation, which we have used in earlier editions: $\text{tf}_{t,d} = \log_{10}(\text{count}(t, d) + 1)$

⁴ *Sweet* was one of Shakespeare's favorite adjectives, a fact probably related to the increased use of sugar in European recipes around the turn of the 16th century (Jurafsky, 2014, p. 175).

11.1.3 Document Scoring

Once we have represented each document and query as a weighted vector, we need to score each document. Our goal is to measure the **relevance** of the document to the user's information need, as expressed in their query. In the classic tf-idf model we estimate this relevance of a document by measuring its geometric similarity in vector space to the query. That is, we make the simplifying assumption that documents that have **similar words** to the query are more relevant to the user.

We use the cosine similarity function introduced in Chapter 5, scoring document d by the cosine of its vector $\mathbf{d}$ with the query vector $\mathbf{q}$:

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|} \quad (11.7)$$

Another way to think of the cosine computation is as the dot product of unit vectors; we can first normalize both the query and document vector to unit vectors, by dividing by their lengths, and then take the dot product:

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}}{|\mathbf{q}|} \cdot \frac{\mathbf{d}}{|\mathbf{d}|} \quad (11.8)$$

We can spell out Eq. 11.8, using the tf-idf values and spelling out the dot product as a sum of products:

$$\text{score}(q, d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}} \quad (11.9)$$

Now let's use Eq. 11.9 to walk through an example of a tiny query against a collection of 4 nano documents, computing tf-idf values and seeing the rank of the documents. We'll assume all words in the following query and documents are down-cased and punctuation is removed:

Query: sweet love
Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

Fig. 11.6 shows the computation of the tf-idf cosine between the query and Document 1, and the query and Document 2. The cosine is the normalized dot product of tf-idf values, so for the normalization we must need to compute the document vector lengths $|\mathbf{q}|$, $|\mathbf{d}_1|$, and $|\mathbf{d}_2|$ for the query and the first two documents using Eq. 11.4, Eq. 11.5, Eq. 11.6, and Eq. 11.9 (computations for Documents 3 and 4 are also needed but are left as an exercise for the reader). The dot product between the vectors is the sum over dimensions of the product, for each dimension, of the values of the two tf-idf vectors for that dimension. This product is only non-zero where both the query and document have non-zero values, so for this example, in which only *sweet* and *love* have non-zero values in the query, the dot product will be the sum of the products of those elements of each vector.

Document 1 has a higher cosine with the query (0.747) than Document 2 has with the query (0.0779), and so the tf-idf cosine model would rank Document 1 above Document 2. This ranking is intuitive given the vector space model, since Document 1 has both terms including two instances of *sweet*, while Document 2 is

Query						
word	cnt	tf	df	idf	tf-idf	n'lized = tf-idf/ q
sweet	1	1	3	0.125	0.125	0.383
nurse	0	0	2	0.301	0	0
love	1	1	2	0.301	0.301	0.924
how	0	0	1	0.602	0	0
sorrow	0	0	1	0.602	0	0
is	0	0	1	0.602	0	0
$ q = \sqrt{.125^2 + .301^2} = .326$						

Document 1						Document 2				
word	cnt	tf	tf-idf	n'lized	$\times q$	cnt	tf	tf-idf	n'lized	$\times q$
sweet	2	1.301	0.163	0.357	0.137	1	1.000	0.125	0.203	0.0779
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	0.610	0	0	0	0	0
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$ d_1 = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$ d_2 = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column: 0.747						Cosine: $\sum$ of column: 0.0779				

Figure 11.6 Computation of tf-idf cosine score between the query and nano-documents 1 (0.747) and 2 (0.0779), using Eq. 11.4, Eq. 11.5, Eq. 11.6 and Eq. 11.9.

missing one of the terms. We leave the computation for Documents 3 and 4 as an exercise for the reader.

In practice, there are many variants and approximations to Eq. 11.9. For example, we might choose to simplify processing by removing some terms. To see this, let's start by expanding the formula for tf-idf in Eq. 11.9 to explicitly mention the tf and idf terms from Eq. 11.6:

$$\text{score}(q, d) = \sum_{t \in q} \frac{\text{tf}_{t,q} \cdot \text{idf}_t}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf}_{t,d} \cdot \text{idf}_t}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}} \quad (11.10)$$

In one common variant of tf-idf cosine, for example, we drop the idf term for the document. Eliminating the second copy of the idf term (since the identical term is already computed for the query) turns out to sometimes result in better performance:

$$\text{score}(q, d) = \sum_{t \in q} \frac{\text{tf}_{t,q} \cdot \text{idf}_t}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf}_{t,d}}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}} \quad (11.11)$$

Other variants of tf-idf eliminate various other terms.

BM25

A slightly more complex variant in the tf-idf family is the **BM25** weighting scheme (sometimes called Okapi BM25 after the Okapi IR system in which it was introduced (Robertson et al., 1995)). BM25 adds two parameters: k , a knob that adjusts the balance between term frequency and IDF, and b , which controls the importance of document length normalization. The BM25 score of a document d given

a query q is:

$$\sum_{t \in q} \overbrace{\log \left(\frac{N}{df_t} \right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k \left(1 - b + b \left(\frac{|d|}{|d_{\text{avg}}|} \right) \right) + tf_{t,d}}}^{\text{weighted tf}} \quad (11.12)$$

where $|d_{\text{avg}}|$ is the length of the average document. When k is 0, BM25 reverts to no use of term frequency, just a binary selection of terms in the query (plus idf). A large k results in raw term frequency (plus idf). b ranges from 1 (scaling by document length) to 0 (no length scaling). Manning et al. (2008) suggest reasonable values are $k = [1.2, 2]$ and $b = 0.75$. Kamphuis et al. (2020) is a useful summary of the many minor variants of BM25.

Stop words In the past it was common to remove high-frequency words from both the query and document before representing them. The list of such high-frequency words to be removed is called a **stop list**. The intuition is that high-frequency terms (often function words like *the*, *a*, *to*) carry little semantic weight and may not help with retrieval, and can also help shrink the inverted index files we describe below. The downside of using a stop list is that it makes it difficult to search for phrases that contain words in the stop list. For example, common stop lists would reduce the phrase *to be or not to be* to the phrase *not*. In modern IR systems, the use of stop lists is much less common, partly due to improved efficiency and partly because much of their function is already handled by IDF weighting, which downweights function words that occur in every document. Nonetheless, stop word removal is occasionally useful in various NLP tasks so is worth keeping in mind.

11.1.4 Efficiently finding documents: the Inverted Index

In order to compute scores, we need to efficiently find documents that contain words in the query. (Any document that contains none of the query terms will have a score of 0 and can be ignored.) The basic search problem in IR is thus to find all documents $d \in C$ that contain a term $q \in Q$.

The data structure for this task is the **inverted index**, which we use for making this search efficient, and also conveniently storing useful information like the document frequency and the count of each term in each document.

An inverted index, given a query term, gives a list of documents that contain the term. It consists of two parts, a **dictionary** and the **postings**. The dictionary is a list of terms (designed to be efficiently accessed), each pointing to a **postings list** for the term. A postings list is the list of document IDs associated with each term, which can also contain information like the term frequency or even the exact positions of terms in the document. The dictionary can also store the document frequency for each term. For example, a simple inverted index for our 4 sample documents above, with each word containing its document frequency in $\{\}$, and a pointer to a postings list that contains document IDs and term counts in $[\]$, might look like the following:

how	{1}	→	3	[1]
is	{1}	→	3	[1]
love	{2}	→	1	[1] → 3 [1]
nurse	{2}	→	1	[1] → 4 [1]
sorrow	{1}	→	2	[1]
sweet	{3}	→	1	[2] → 2 [1] → 3 [1]

Given a list of terms in query, we can very efficiently get lists of all candidate documents, together with the information necessary to compute the tf-idf scores we need.

11.2 Evaluation of Information-Retrieval Systems

We measure the performance of ranked retrieval systems using the same **precision** and **recall** metrics we have been using. We make the assumption that each document returned by the IR system is either **relevant** to our purposes or **not relevant**. Precision is the fraction of the returned documents that are relevant, and recall is the fraction of all relevant documents that are returned. More formally, let's assume a system returns T ranked documents in response to an information request, a subset R of these are relevant, a disjoint subset, N , are the remaining irrelevant documents, and U documents in the collection as a whole are relevant to this request. Precision and recall are then defined as:

$$Precision = \frac{|R|}{|T|} \quad Recall = \frac{|R|}{|U|} \quad (11.13)$$

Unfortunately, these metrics don't adequately measure the performance of a system that *rank*s the documents it returns. If we are comparing the performance of two ranked retrieval systems, we need a metric that prefers the one that ranks the relevant documents higher. We need to adapt precision and recall to capture how well a system does at putting relevant documents higher in the ranking.

Let's turn to an example. Assume the table in Fig. 11.7 gives rank-specific precision and recall values calculated as we proceed down through a set of ranked documents for a particular query; the precisions are the fraction of relevant documents seen at a given rank, and recalls the fraction of relevant documents found at the same rank. The recall measures in this example are based on this query having 9 relevant documents in the collection as a whole.

Note that recall is non-decreasing; when a relevant document is encountered, recall increases, and when a non-relevant document is found it remains unchanged. Precision, on the other hand, jumps up and down, increasing when relevant documents are found, and decreasing otherwise. The most common way to visualize precision and recall is to plot precision against recall in a **precision-recall curve**, like the one shown in Fig. 11.8 for the data in table 11.7.

Fig. 11.8 shows the values for a single query. But we'll need to combine values for all the queries, and in a way that lets us compare one system to another. One way of doing this is to plot averaged precision values at 11 fixed levels of recall (0 to 100, in steps of 10). Since we're not likely to have datapoints at these exact levels, we use **interpolated precision** values for the 11 recall values from the data points we do have. We can accomplish this by choosing the maximum precision value achieved at any level of recall at or above the one we're calculating. In other words,

$$\text{IntPrecision}(r) = \max_{i \geq r} \text{Precision}(i) \quad (11.14)$$

This interpolation scheme not only lets us average performance over a set of queries, but also helps smooth over the irregular precision values in the original data. It is designed to give systems the benefit of the doubt by assigning the maximum precision value achieved at higher levels of recall from the one being measured. Fig. 11.9 and Fig. 11.10 show the resulting interpolated data points from our example.